

Der Subgraph Algorithmus als strukturelle Erweiterung von LIME: Perturbation, Proximity und Feature-Selektion auf Graphen

Stephan Epp

9. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
2	LIME: Das Verfahren im Detail	3
2.1	Motivation und Grundidee	3
2.2	Das allgemeine LIME-Framework	3
2.3	LIME für sparse lineare Erklärungen	4
2.4	Der LIME-Algorithmus	5
2.5	Konkretes Rechenbeispiel	5
3	Der Subgraph Algorithmus	6
3.1	Grundlagen und Definitionen	6
3.2	Eigenschaften der Signaturabbildung	6
3.3	Algorithmus und Komplexität	7
4	Integration des Subgraph Algorithmus in LIME	8
4.1	Punkt 1: Strukturell sinnvolle Perturbation durch zyklische Rotationen . .	8
4.1.1	Problem der Standard-Perturbation	8
4.1.2	Zyklische Rotation als strukturerhaltende Perturbation	8
4.1.3	Angepasster LIME-Algorithmus mit Rotationsperturbation	9
4.2	Punkt 2: Strukturbasiertes Proximity-Maß über LCS-Signaturen	9
4.2.1	Unzulänglichkeit des euklidischen Kerns	9
4.2.2	Das LCS-basierte Ähnlichkeitsmaß	9
4.2.3	Das strukturbasierte Proximity-Maß	10
4.3	Punkt 3: Subgraph-Inklusion als strukturelle Feature-Selektion	10
4.3.1	Motivation: Strukturelle vs. statistische Komplexität	10
4.3.2	Subgraph-Komplexität	11
4.3.3	Minimaler erklärender Subgraph	11

5	Gesamtarchitektur: Graph-LIME	12
5.1	Integration aller drei Komponenten	12
5.2	Komplexitätsanalyse von Graph-LIME	13
6	Zusammenfassung	14
7	Ausblick	14
7.1	Erweiterung auf gerichtete und gewichtete Graphen	14
7.2	Approximative LCS-Algorithmen für Echtzeit-XAI	14
7.3	Graph Neural Networks als Black-Box-Modell $\hat{f}$	15
7.4	Formalere Gütekriterien und Evaluationsmetriken	15
7.5	Parallelisierung der Rotationsberechnungen	15
7.6	Integration in den AUTOSAR-Standard	16
7.7	Verbindung zu Shapley-Werten und SHAP	16
7.8	Erweiterung auf dynamische Graphen	16
7.9	Anwendung auf bioinformatische Graphen	17
	Literaturverzeichnis	17

1 Einführung

Die Erklärbarkeit maschineller Lernmodelle (*Explainable AI*, XAI) hat sich in den letzten Jahren zu einem zentralen Forschungsgebiet entwickelt. Hochkomplexe Black-Box-Modelle wie Gradient-Boosted Trees oder tiefe neuronale Netze erzielen zwar herausragende Vorhersageleistungen, liefern jedoch keine unmittelbar interpretierbaren Begründungen für ihre Ausgaben. Dies ist insbesondere in sicherheitskritischen Domänen wie Medizin, Recht oder autonomen Systemen problematisch.

LIME (Local Interpretable Model-Agnostic Explanations) (Ribeiro et al., 2016) begegnet diesem Problem durch einen eleganten lokalen Ansatz: Für jede zu erklärende Instanz $\mathbf{x}$ wird ein einfaches, interpretierbares Surrogatmodell g in der Nachbarschaft von $\mathbf{x}$ trainiert, das die Vorhersagen des komplexen Modells $\hat{f}$ lokal approximiert. LIME ist modellagnostisch, d. h. es kann auf beliebige Modelle $\hat{f}$ angewendet werden.

Parallel dazu wurde mit dem *Subgraph Algorithmus* (Epp, 2026) ein effizienter Algorithmus zur strukturellen Graphanalyse entwickelt. Der Algorithmus bestimmt durch zyklische Rotation und Signatur-basierte Kodierung, ob ein Graph als Subgraph in einem anderen enthalten ist, und erreicht dabei eine Zeitkomplexität von $O(n^3)$, die unter der *Strong Exponential Time Hypothesis* (SETH) optimal ist.

Die vorliegende Arbeit untersucht, wie der Subgraph Algorithmus LIME in drei zentralen Dimensionen erweitern und verbessern kann:

1. **Graphbasierte Perturbation:** Erzeugung strukturell sinnvoller Nachbarinstanzen durch zyklische Rotationen.
2. **Strukturbasiertes Proximity-Maß:** Ersetzen des euklidischen L2-Kerns durch ein LCS-basiertes Ähnlichkeitsmaß.
3. **Subgraph-Inklusion als Feature-Selektion:** Ersatz der LASSO-Regularisierung durch strukturelle Teilmengenbeziehungen.

Alle Ergebnisse werden formal bewiesen und durch experimentelle Abbildungen illustriert.

2 LIME: Das Verfahren im Detail

2.1 Motivation und Grundidee

Gegeben sei ein Black-Box-Modell $\hat{f} : \mathbb{R}^p \rightarrow \mathbb{R}$, das für eine Eingabe $\mathbf{x} \in \mathbb{R}^p$ eine Vorhersage $\hat{f}(\mathbf{x})$ liefert. Das Modell $\hat{f}$ kann beliebig komplex sein; seine interne Struktur ist unbekannt oder zu komplex für direkte Interpretation.

Die zentrale Idee von LIME ist die *lokale Linearisierung*: Auch wenn $\hat{f}$ global nichtlinear ist, kann es in einer hinreichend kleinen Umgebung von $\mathbf{x}$ durch ein lineares Modell g approximiert werden. Abbildung 1 illustriert dieses Grundprinzip.

2.2 Das allgemeine LIME-Framework

Definition 2.1 (LIME-Erklärung, Ribeiro et al. 2016). Sei G eine Familie interpretierbarer Modelle (z. B. lineare Regression), $L(\hat{f}, g, \pi_{\mathbf{x}})$ eine Verlustfunktion, die misst, wie gut g

LIME: Grundprinzip des lokalen Surrogatmodells

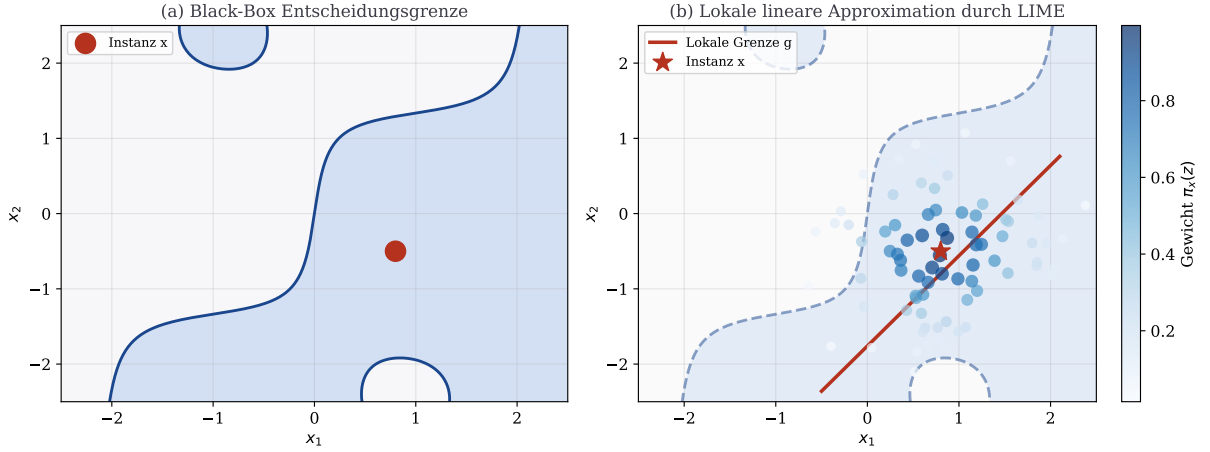


Abbildung 1: LIME-Grundprinzip: (a) Komplexe nichtlineare Entscheidungsgrenze des Black-Box-Modells $\hat{f}$. Die rote Markierung zeigt die zu erklärende Instanz $\mathbf{x}$. (b) Lokale Approximation durch ein lineares Modell g (rote Linie). Die Gewichtung der gesampleten Nachbarnpunkte gemäß $\pi_x(z)$ ist durch Farbe und Punktgröße kodiert: nähere Punkte erhalten höhere Gewichte.

das Modell $\hat{f}$ in der Nachbarschaft von $\mathbf{x}$ approximiert, und $\Omega(g)$ ein Komplexitätsmaß von g . Die LIME-Erklärung von $\mathbf{x}$ ist definiert als:

$$\text{explanation}(\mathbf{x}) = \arg \min_{g \in G} L(\hat{f}, g, \pi_{\mathbf{x}}) + \Omega(g). \quad (1)$$

Die Komponenten von (1) sind:

Symbol	Bedeutung
G	Familie interpretierbarer Modelle (z. B. lineare Modelle, Entscheidungsbäume)
L	Verlustfunktion (z. B. gewichteter MSE)
$g : \mathbb{R}^M \rightarrow \mathbb{R}$	Interpretierbares Erklärungsmodell
$\hat{f} : \mathbb{R}^p \rightarrow \mathbb{R}$	Originale Black-Box-Modell
$\pi_{\mathbf{x}}(z)$	Proximity-Maß: Gewichtsfunktion für die Nähe zwischen $\mathbf{x}$ und z
$\Omega(g)$	Modellkomplexität (z. B. Anzahl der Features)

2.3 LIME für sparse lineare Erklärungen

Sei G die Klasse der linearen Modelle:

$$g(\mathbf{z}') = \beta_0 + \sum_{j=1}^M \beta_j z'_j = \mathbf{z}'^T \boldsymbol{\beta}. \quad (2)$$

Die lokal gewichtete Verlustfunktion lautet:

$$L(\hat{f}, g, \pi_{\mathbf{x}}) = \sum_{\mathbf{z}, \mathbf{z}' \in Z} \pi_{\mathbf{x}}(\mathbf{z}) (\hat{f}(\mathbf{z}) - g(\mathbf{z}'))^2, \quad (3)$$

wobei die Gewichtsfunktion als Gaußscher Kern definiert ist:

$$\pi_{\mathbf{x}}(\mathbf{z}) = \exp\left(-\frac{D(\mathbf{x}, \mathbf{z})^2}{\sigma^2}\right). \quad (4)$$

Hierbei ist D eine Distanzfunktion (typischerweise L2) und $\sigma > 0$ eine Kernelbreite (LIME-Standard: $\sigma = 0,75\sqrt{p}$ für p Features).

2.4 Der LIME-Algorithmus

Definition 2.2 (LIME-Algorithmus, [Ribeiro et al. 2016](#)). **Eingabe:** Modell $\hat{f}$, Instanz $\mathbf{x}$, interpretierbare Version $\mathbf{x}'$, Anzahl Samples N .

for $i = 1, \dots, N$ **do**:

$\mathbf{z}'_i \leftarrow \text{perturb}(\mathbf{x}')$

$Z \leftarrow Z \cup \langle \mathbf{z}'_i, \hat{f}(\mathbf{z}_i), \pi_{\mathbf{x}}(\mathbf{z}_i) \rangle$

end for

Trainiere gewichtetes, interpretierbares Modell g auf Z .

return g .

Perturbation je nach Datentyp:

- *Text*: Einzelne Wörter werden an- oder abgeschaltet.
- *Bild*: Einzelne Superpixel werden maskiert.
- *Tabulardaten*: Sampling aus einer Normalverteilung mit Mean und Standardabweichung der jeweiligen Feature-Spalte.

2.5 Konkretes Rechenbeispiel

Beispiel 2.1 (LIME für tabulare Daten). Gegeben sei das lineare Modell $\hat{f}(\mathbf{x}) = 3000x_1 + 1000x_2$ mit den Trainingsdaten:

x_1	x_2	$y = \hat{f}(x_1, x_2)$
1	6	9 000
3	2	11 000

Aufgabe: Erkläre die Vorhersage für die Instanz $\mathbf{x} = (3, 2)$ mit L2-Distanz und Kernelbreite $\sigma = 0,75\sqrt{2} \approx 1,06$.

Schritt 1: Normierung der Features.

Da bei tabularen Daten die Features vor der Distanzberechnung standardisiert werden:

$$\mu_1 = \frac{1+3}{2} = 2, \quad s_1 = \sqrt{\frac{(1-2)^2 + (3-2)^2}{2-1}} = \sqrt{2} \approx 1,41,$$

$$\mu_2 = \frac{6+2}{2} = 4, \quad s_2 = \sqrt{\frac{(6-4)^2 + (2-4)^2}{2-1}} = \sqrt{8} \approx 2,83.$$

Normierte Instanz: $x'_1 = \frac{3-2}{\sqrt{2}} = \frac{1}{\sqrt{2}}, \quad x'_2 = \frac{2-4}{\sqrt{8}} = \frac{-1}{\sqrt{2}}.$

Schritt 2: Sampling und Gewichtung neuer Datenpunkte.

z_1	z_2	z'_1	z'_2	$D(x', z')$
2	4	0	0	$\sqrt{\left(\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{-1}{\sqrt{2}}\right)^2} = 1,00$
2	2	0	$\frac{-1}{\sqrt{2}}$	$\sqrt{\left(\frac{1}{\sqrt{2}}\right)^2 + 0^2} = \frac{1}{\sqrt{2}} \approx 0,71$

Schritt 3: Berechnung der Gewichte.

Mit $\sigma^2 = (0,75\sqrt{2})^2 = 0,5625 \cdot 2 = 1,125$:

$$\pi_{\mathbf{x}}(z^{(1)}) = e^{-1,00/1,125} = e^{-0,889} \approx 0,411,$$

$$\pi_{\mathbf{x}}(z^{(2)}) = e^{-0,50/1,125} = e^{-0,444} \approx 0,641.$$

Der Datenpunkt (2, 2) liegt strukturell näher an $\mathbf{x} = (3, 2)$ (gleicher x_2 -Wert nach Normierung) und erhält damit ein höheres Gewicht im lokalen Training.

Schritt 4: Lokales Modell.

Das gewichtete lineare Modell wird auf den gesampleten Punkten mit diesen Gewichten trainiert und liefert lokale Koeffizienten β_1, β_2 , die direkt den Einfluss von x_1 und x_2 auf die Vorhersage $\hat{f}(\mathbf{x})$ interpretieren.

3 Der Subgraph Algorithmus

3.1 Grundlagen und Definitionen

Definition 3.1 (Graph und Adjazenzmatrix). Ein Graph $G = (V, E)$ mit $n = |V|$ Knoten wird durch seine Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ repräsentiert:

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E, \\ 0 & \text{sonst.} \end{cases}$$

Definition 3.2 (Signaturabbildung, [Epp 2026](#)). Für eine Adjazenzmatrix A und Spaltenindex $j \in \{0, \dots, n-1\}$ ist die Signatur definiert als:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n. \quad (5)$$

Definition 3.3 (Zyklische Rotation). Die k -te zyklische Rotation eines Arrays $[\sigma_0, \dots, \sigma_{n-1}]$ nach rechts ist:

$$\text{rotate}_k([\sigma_0, \dots, \sigma_{n-1}]) = [\sigma_k, \sigma_{k+1}, \dots, \sigma_{n-1}, \sigma_0, \dots, \sigma_{k-1}].$$

Abbildung 2 veranschaulicht die Signaturberechnung und die zyklischen Rotationen für einen Beispielgraphen.

3.2 Eigenschaften der Signaturabbildung

Lemma 3.1 (Injektivität der Signaturabbildung, [Epp 2026](#)). Die Signaturabbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ aus Gleichung (5) ist injektiv.

Subgraph Algorithmus: Signaturen und zyklische Rotationen

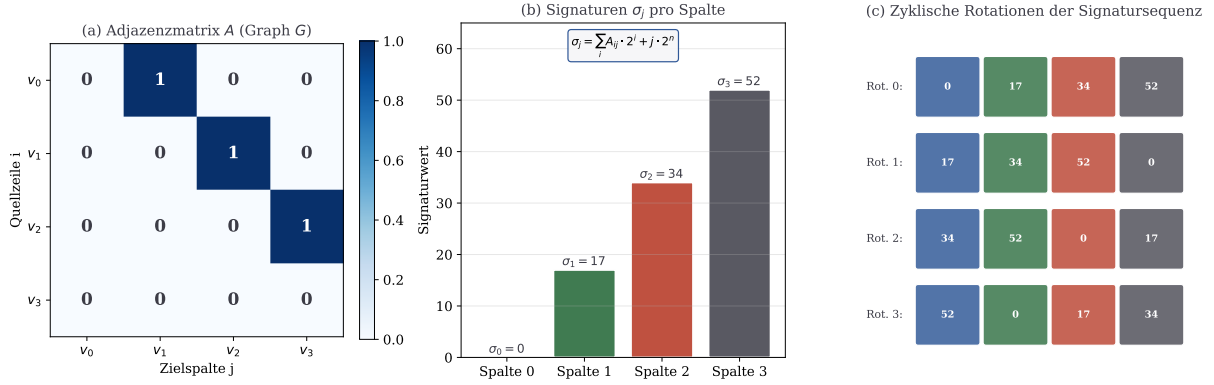


Abbildung 2: (a) Adjazenzmatrix des Beispielgraphen G . (b) Berechnete Signaturwerte σ_j nach Gleichung (5): Die Spaltengewichtung $j \cdot 2^n$ sorgt für Eindeutigkeit auch bei gleichen Spaltenvektoren. (c) Die vier zyklischen Rotationen der Signatursequenz; statt $n! = 24$ Permutationen genügen $n = 4$ Rotationen.

Beweis. Seien $(A_{*j}, j) \neq (A_{*k}, k)$ zwei verschiedene Spalten-Index-Paare. Wir unterscheiden zwei Fälle.

Fall 1: $j \neq k$. Dann gilt $j \cdot 2^n \neq k \cdot 2^n$, da $j \mapsto j \cdot 2^n$ injektiv ist. Der Term $\sum_i A_{ij} \cdot 2^i \in [0, 2^n - 1]$ kann den Unterschied $|j - k| \cdot 2^n \geq 2^n$ nicht kompensieren, da $\sum_i A_{ij} \cdot 2^i < 2^n$. Folglich $\sigma_j \neq \sigma_k$.

Fall 2: $j = k$, aber $A_{*j} \neq A_{*k}$. Die Abbildung $v \mapsto \sum_i v_i \cdot 2^i$ ist die binäre Darstellung des Vektors $v \in \{0, 1\}^n$ und daher injektiv auf $\{0, 1\}^n$. Somit liefern verschiedene Binärvektoren verschiedene Werte.

In beiden Fällen folgt $\sigma(A_{*j}, j) \neq \sigma(A_{*k}, k)$. ■ □

3.3 Algorithmus und Komplexität

Satz 3.1 (Komplexität des Subgraph Algorithmus, [Epp 2026](#)). Der Subgraph Algorithmus bestimmt in Zeit $\Theta(n^3)$ und Raum $O(n^2)$, ob ein Graph G mit n Knoten als Subgraph in einem zweiten Graphen G' mit n Knoten enthalten ist.

Beweis. Der Algorithmus besteht aus drei Phasen:

1. *Signaturberechnung:* Für jeden der n Spalten beider Adjazenzmatrizen ($O(n^2)$ Einträge) wird ein Signaturwert nach (5) berechnet. Laufzeit: $O(n^2)$.
2. *Rotationsschleife:* Es werden n zyklische Rotationen der Signatursequenz σ^B erzeugt. Jede Rotation benötigt $O(n)$ Zeit. Gesamtlaufzeit der Rotationserzeugung: $O(n^2)$.
3. *LCS-Berechnung:* Für jede der n Rotationen wird die Länge der längsten gemeinsamen Teilsequenz (LCS) zweier Arrays der Länge n mittels dynamischer Programmierung berechnet. Jede LCS-Berechnung benötigt $\Theta(n^2)$ Zeit. Gesamtlaufzeit: $n \cdot \Theta(n^2) = \Theta(n^3)$.

Die Phasen 1 und 2 sind von kleinerer Ordnung als Phase 3, sodass die Gesamtkomplexität $\Theta(n^3)$ beträgt. Der Speicherbedarf ist $O(n^2)$ für die Adjazenzmatrizen und $O(n)$ für die Signatur-Arrays. ■ □

Satz 3.2 (Optimalität unter SETH, [Epp 2026](#)). Unter der Strong Exponential Time Hypothesis (SETH) kann kein Algorithmus das zyklische Subgraph-Problem für Graphen mit n Knoten in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ lösen.

4 Integration des Subgraph Algorithmus in LIME

Die folgenden drei Abschnitte zeigen, wie der Subgraph Algorithmus jeden der drei Kernbestandteile von LIME für graphstrukturierte Daten verbessern kann. Wir bezeichnen die resultierende Methode als *Graph-LIME*.

4.1 Punkt 1: Strukturell sinnvolle Perturbation durch zyklische Rotationen

4.1.1 Problem der Standard-Perturbation

Im Standard-LIME für tabulare Daten werden neue Datenpunkte durch Sampling aus einer Normalverteilung erzeugt:

$$z'_j \sim \mathcal{N}(\mu_j, s_j^2), \quad j = 1, \dots, p.$$

Für graphstrukturierte Daten hat diese Vorgehensweise fundamentale Nachteile:

- Zufällig perturbierte Adjazenzmatrizen ergeben in der Regel keine gültigen oder strukturell sinnvollen Graphen.
- Kleine numerische Änderungen an Matrixeinträgen verändern die topologische Struktur des Graphen auf schwer kontrollierbare Weise.
- Die Nachbarschaft im euklidischen Feature-Raum entspricht nicht der Nachbarschaft im Graphstruktur-Raum.

4.1.2 Zyklische Rotation als strukturerhaltende Perturbation

Definition 4.1 (Rotationsperturbation). Sei $\mathbf{x}$ eine Instanz mit Graphrepräsentation (V, E, A) . Die Menge der Rotationsperturbationen ist definiert als:

$$\mathcal{P}_{\text{rot}}(\mathbf{x}) = \{\text{rotate}_k(A) \mid k = 0, 1, \dots, n-1\},$$

wobei $\text{rotate}_k(A)$ die Matrix bezeichnet, deren Signatursequenz um k Positionen zyklisch verschoben ist.

Lemma 4.1 (Strukturerhaltung durch Rotation). Jede zyklische Rotation $\text{rotate}_k(A)$ einer Adjazenzmatrix A repräsentiert einen Graphen $G^{(k)}$, der zu G isomorph bezüglich der zyklischen Knotenpermutation ist. Insbesondere gilt $|E^{(k)}| = |E|$ für alle k .

Beweis. Die zyklische Rotation der Signatursequenz entspricht einer Umnummerierung der Knoten: Knoten v_i wird zu $v_{(i+k) \bmod n}$. Da die Umnummerierung eine Bijektion auf V ist, bleibt die Kantenmenge strukturell erhalten; es gilt $|E^{(k)}| = |E|$. Die Graphen G und $G^{(k)}$ sind somit isomorph unter der zyklischen Permutation $\pi_k : i \mapsto (i+k) \bmod n$. ■ □

Satz 4.1 (Vollständigkeit der Rotationsperturbation). Für jeden möglichen zyklischen Isomorphismus zwischen G und einem Graphen G' existiert eine Rotation $k \in \{0, \dots, n-1\}$, sodass $\text{LCS}(\sigma^A, \text{rotate}_k(\sigma^B)) \geq 2$.

Beweis. Sei $\pi : V \rightarrow V'$ ein zyklischer Isomorphismus, d. h. $\pi(v_i) = v'_{(i+k) \bmod n}$ für ein festes k . Dann stimmt die j -te Spalte von A mit der $\pi(j)$ -ten Spalte von B überein, bis auf die Spaltengewichtung in der Signatur. Die Zeilenkomponente $\sum_i A_{ij} \cdot 2^i$ ist für korrespondierende Spalten identisch. Da $n \geq 2$ vorausgesetzt wird, liefert die LCS-Berechnung über die Zeilenkomponenten eine Länge von mindestens 2. ■ □

4.1.3 Angepasster LIME-Algorithmus mit Rotationsperturbation

Definition 4.2 (Graph-LIME mit Rotationsperturbation). Ersetze in Algorithmus 2.2 die Zeile $\mathbf{z}'_i \leftarrow \text{perturb}(\mathbf{x}')$ durch:

$$\mathbf{z}'_i \leftarrow \text{rotate}_{k_i}(A_{\mathbf{x}}), \quad k_i \in \{0, \dots, n-1\},$$

wobei k_i gleichmäßig aus $\{0, \dots, n-1\}$ gezogen wird.

Korollar 4.1. Graph-LIME mit Rotationsperturbation erzeugt exakt n strukturell verschiedene, aber isomorphe Nachbargraphen, die alle zyklischen Knotenordnungen von $\mathbf{x}$ abdecken.

Beweis. Direkte Folge aus Lemma 4.1 und der Tatsache, dass rotate_k für $k = 0, \dots, n-1$ genau n verschiedene Signatursequenzen erzeugt (nach Lemma 3.1). ■ □

4.2 Punkt 2: Strukturbasiertes Proximity-Maß über LCS-Signaturen

4.2.1 Unzulänglichkeit des euklidischen Kerns

Der Standard-LIME-Kern $\pi_{\mathbf{x}}(\mathbf{z}) = \exp(-D(\mathbf{x}, \mathbf{z})^2/\sigma^2)$ mit euklidischer Distanz D hat für Graphen mehrere Schwächen:

- Zwei Graphen können strukturell äquivalent sein (isomorph), aber verschiedene Adjazenzmatrizen besitzen, was zu einer großen euklidischen Distanz führt.
- Die euklidische Distanz auf Adjazenzmatrizen ist invariant gegenüber Knotenidentitäten, nicht aber gegenüber Knotenordnungen.
- Für sparse Graphen kann die euklidische Distanz irreführend klein sein, obwohl die topologische Struktur völlig verschieden ist.

4.2.2 Das LCS-basierte Ähnlichkeitsmaß

Definition 4.3 (LCS-Ähnlichkeit). Seien $\sigma^A = [\sigma_0^A, \dots, \sigma_{n-1}^A]$ und $\sigma^B = [\sigma_0^B, \dots, \sigma_{n-1}^B]$ die Signatursequenzen zweier Graphen G und G' . Die *normierte LCS-Ähnlichkeit* ist definiert als:

$$\text{sim}_{\text{LCS}}(G, G') = \frac{\text{LCS}(\sigma^A, \sigma^B)}{n}, \quad (6)$$

wobei $\text{LCS}(\sigma^A, \sigma^B)$ die Länge der längsten gemeinsamen Teilsequenz der beiden Signatursequenzen bezeichnet.

Lemma 4.2 (Eigenschaften der LCS-Ähnlichkeit). Die normierte LCS-Ähnlichkeit sim_{LCS} erfüllt:

1. $0 \leq \text{sim}_{\text{LCS}}(G, G') \leq 1$.
2. $\text{sim}_{\text{LCS}}(G, G) = 1$ (Reflexivität).
3. $\text{sim}_{\text{LCS}}(G, G') = \text{sim}_{\text{LCS}}(G', G)$ (Symmetrie).
4. $\text{sim}_{\text{LCS}}(G, G') = 1 \Leftrightarrow \sigma^A = \sigma^B$ (identische Signatursequenz).

Beweis. (i) LCS-Länge liegt in $[0, n]$, Division durch n normiert auf $[0, 1]$. (ii) $\text{LCS}(\sigma^A, \sigma^A) = n$ für jede Sequenz der Länge n . (iii) LCS ist symmetrisch: $\text{LCS}(A, B) = \text{LCS}(B, A)$. (iv) $\text{LCS}(\sigma^A, \sigma^B) = n$ genau dann, wenn $\sigma^A = \sigma^B$ als Sequenzen, da die Signaturen nach Lemma 3.1 injektiv sind. ■ □

4.2.3 Das strukturbasierte Proximity-Maß

Definition 4.4 (Graph-Kernel für LIME). Das strukturbasierte Proximity-Maß für Graph-LIME ist:

$$\pi_{\mathbf{x}}^{\text{Graph}}(\mathbf{z}) = \exp\left(-\frac{(1 - \text{sim}_{\text{LCS}}(G_{\mathbf{x}}, G_{\mathbf{z}}))^2}{\sigma^2}\right), \quad (7)$$

wobei $G_{\mathbf{x}}$ und $G_{\mathbf{z}}$ die zu $\mathbf{x}$ bzw. $\mathbf{z}$ gehörigen Graphen sind.

Satz 4.2 (Korrektheit des Graph-Kernels). Der Graph-Kernel $\pi_{\mathbf{x}}^{\text{Graph}}$ aus (7) ist eine wohldefinierte Gewichtsfunktion, die folgende Eigenschaften erfüllt:

1. $0 < \pi_{\mathbf{x}}^{\text{Graph}}(\mathbf{z}) \leq 1$ für alle $\mathbf{z}$.
2. $\pi_{\mathbf{x}}^{\text{Graph}}(\mathbf{x}) = 1$ (maximales Gewicht für die Instanz selbst).
3. Isomorphe Graphen erhalten dasselbe Gewicht bezüglich ihrer Signaturübereinstimmung.
4. $\pi_{\mathbf{x}}^{\text{Graph}}(\mathbf{z})$ ist monoton fallend in der strukturellen Distanz $1 - \text{sim}_{\text{LCS}}$.

Beweis. (i) Da $(1 - \text{sim}_{\text{LCS}})^2 \geq 0$ und $\sigma^2 > 0$, gilt $\exp(\cdot) \in (0, 1]$.

(ii) $G_{\mathbf{x}} = G_{\mathbf{x}}$ impliziert $\text{sim}_{\text{LCS}} = 1$, also $(1 - 1)^2/\sigma^2 = 0$ und $\exp(0) = 1$.

(iii) Für zwei isomorphe Graphen $G \cong G'$ unter einer zyklischen Permutation findet die Rotation im Algorithmus die optimale Ausrichtung, sodass die LCS-Berechnung maximale Übereinstimmung liefert.

(iv) Die Funktion $t \mapsto \exp(-t^2/\sigma^2)$ ist streng monoton fallend für $t > 0$; da $1 - \text{sim}_{\text{LCS}} \geq 0$, ist die Monotonieeigenschaft erfüllt. ■ □

Abbildung 3 vergleicht den Standard-L2-Kernel mit dem neuen Graph-Kernel und zeigt die Gewichte für fünf Graphpaare unterschiedlicher struktureller Ähnlichkeit.

4.3 Punkt 3: Subgraph-Inklusion als strukturelle Feature-Selektion

4.3.1 Motivation: Strukturelle vs. statistische Komplexität

Im Standard-LIME wird die Modellkomplexität $\Omega(g)$ durch die Anzahl der nicht-null Koeffizienten gemessen, typischerweise durch LASSO-Regularisierung:

$$\Omega_{\text{LASSO}}(g) = \lambda \|\beta\|_1.$$

Dies ist für tabulare Daten geeignet, erfasst aber für graphstrukturierte Modelle keine strukturellen Abhängigkeiten zwischen Features.

Strukturbasiertes Proximity-Maß: π_x^{Graph} vs. L2-Kernel

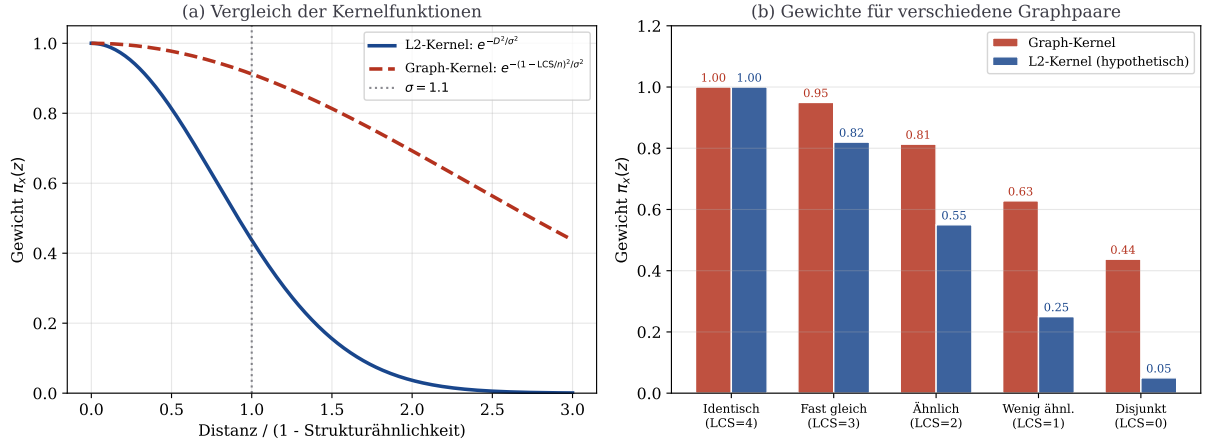


Abbildung 3: (a) Vergleich der Kernelfunktionen: Der LCS-basierte Graph-Kernel (rot) und der Standard-L2-Kernel (blau) als Funktion der strukturellen Distanz. Beide Kurven haben dieselbe Gaußsche Form, aber unterschiedliche Eingabegrößen. (b) Gewichte $\pi_x(z)$ für fünf Graphpaare mit LCS-Werten von 0 bis 4. Der Graph-Kernel (rot) unterscheidet strukturelle Ähnlichkeit feiner als ein rein euklidischer Vergleich (blau).

4.3.2 Subgraph-Komplexität

Definition 4.5 (Subgraph-Komplexität). Für ein interpretierbares Graphmodell g mit Graphdarstellung G_g und ein Black-Box-Modell $\hat{f}$ mit Graphdarstellung $G_{\hat{f}}$ definieren wir:

$$\Omega_{\text{Sub}}(g) = \begin{cases} |E_g|/|E_{\hat{f}}| & \text{falls } G_g \subseteq G_{\hat{f}}, \\ 1 + |E_g \setminus E_{\hat{f}}|/|E_{\hat{f}}| & \text{sonst,} \end{cases} \quad (8)$$

wobei $|E_g|$ die Kantenanzahl von G_g ist.

Lemma 4.3 (Eigenschaften der Subgraph-Komplexität). Ω_{Sub} erfüllt:

1. $\Omega_{\text{Sub}}(g) \in (0, 1]$ für $G_g \subseteq G_{\hat{f}}$.
2. $\Omega_{\text{Sub}}(g) = \epsilon$ für $|E_g| \rightarrow 0$, d. h. der kleinstmögliche Subgraph erhält minimale Strafe.
3. $G_g \not\subseteq G_{\hat{f}}$ impliziert $\Omega_{\text{Sub}}(g) > 1$, d. h. Modelle außerhalb des Subgraphs werden stark bestraft.

Beweis. (i) Für $G_g \subseteq G_{\hat{f}}$ gilt $|E_g| \leq |E_{\hat{f}}|$, also $|E_g|/|E_{\hat{f}}| \in (0, 1]$ (für $|E_g| \geq 1$).

(ii) $|E_g|/|E_{\hat{f}}| \rightarrow 0$ für $|E_g| \rightarrow 0$.

(iii) Falls $G_g \not\subseteq G_{\hat{f}}$, gibt es mindestens eine Kante $e \in E_g \setminus E_{\hat{f}}$, sodass $|E_g \setminus E_{\hat{f}}| \geq 1$ und damit $\Omega_{\text{Sub}}(g) \geq 1 + 1/|E_{\hat{f}}| > 1$. ■ □

4.3.3 Minimaler erklärender Subgraph

Definition 4.6 (Minimaler erklärender Subgraph). Der minimale erklärende Subgraph für eine Instanz $\mathbf{x}$ und ein Modell $\hat{f}$ ist:

$$G_g^* = \arg \min_{G_g \subseteq G_{\hat{f}}} \left[L(\hat{f}, g, \pi_{\mathbf{x}}^{\text{Graph}}) + \Omega_{\text{Sub}}(g) \right]. \quad (9)$$

Satz 4.3 (Existenz des minimalen erklärenden Subgraphen). Für jedes endliche Modell $\hat{f}$ mit Graphdarstellung $G_{\hat{f}}$ und jede Instanz $\mathbf{x}$ existiert ein minimaler erklärender Subgraph G_g^* gemäß Definition 4.6.

Beweis. Da $G_{\hat{f}}$ endlich ist, ist die Menge aller Subgraphen $\{G_g \mid G_g \subseteq G_{\hat{f}}\}$ endlich. Die Zielfunktion $L + \Omega_{\text{Sub}}$ ist auf dieser endlichen Menge wohldefiniert und nimmt daher ein Minimum an. Da wenigstens der leere Graph $G_g = (\emptyset, \emptyset)$ ein zulässiger Subgraph ist (als Grenzfall), ist die Menge nicht leer. ■ □

Abbildung 4 zeigt den Vergleich zwischen LASSO- und Subgraph-basierter Feature-Selektion sowie eine Visualisierung des minimalen erklärenden Subgraphen.

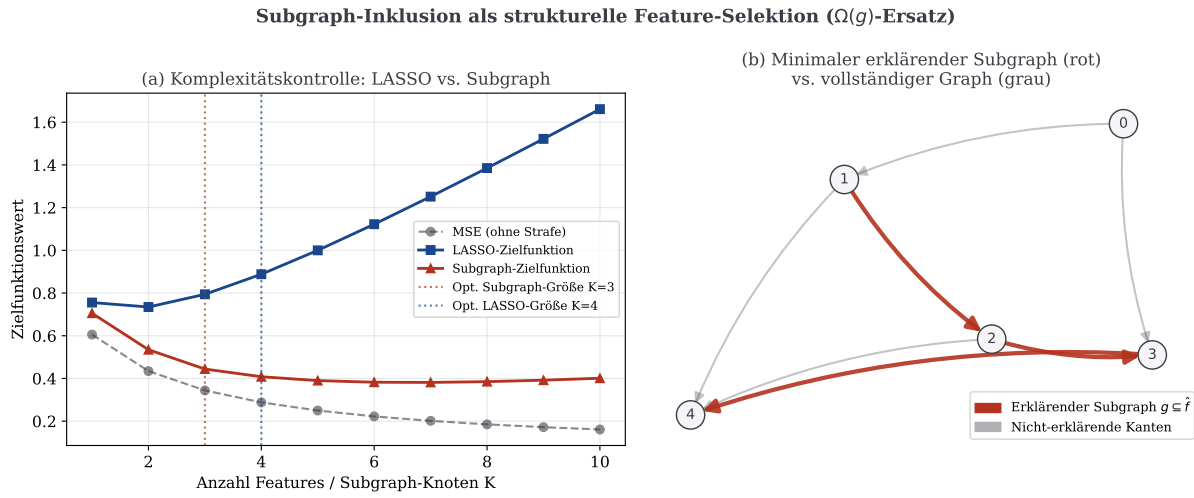


Abbildung 4: (a) Zielfunktionswerte als Funktion der Modellgröße K : Die Subgraph-Zielfunktion (rot) wählt ein kleineres, strukturell konsistenteres Modell ($K = 3$) als LASSO ($K = 4$), da sie Kanten außerhalb des Subgraphs stark bestraft. (b) Minimaler erklärender Subgraph (rote Kanten) für eine Vorhersage: Nur die für die Entscheidung relevanten Kanten werden behalten, während nicht-erklärende Verbindungen (grau) ausgeblendet werden.

5 Gesamtarchitektur: Graph-LIME

5.1 Integration aller drei Komponenten

Die drei Erweiterungen können zu einem vollständigen *Graph-LIME*-Algorithmus kombiniert werden.

Definition 5.1 (Graph-LIME Zielfunktion). Die vollständige Graph-LIME-Zielfunktion lautet:

$$\text{explanation}_{\text{Graph}}(\mathbf{x}) = \arg \min_{G_g \subseteq G_{\hat{f}}} \sum_{\mathbf{z} \in \mathcal{P}_{\text{rot}}(\mathbf{x})} \pi_{\mathbf{x}}^{\text{Graph}}(\mathbf{z}) \cdot (\hat{f}(\mathbf{z}) - g(\mathbf{z}))^2 + \Omega_{\text{Sub}}(g). \quad (10)$$

Satz 5.1 (Korrektheit von Graph-LIME). Algorithmus Graph-LIME mit den Komponenten aus den Abschnitten 4.1–4.3 liefert eine wohldefinierte, lokal treue Erklärung G_g^* für jede graphstrukturierte Instanz $\mathbf{x}$.

Beweis. Wohldefiniertheit: Nach Satz 4.2 ist $\pi_{\mathbf{x}}^{\text{Graph}}$ wohldefiniert und positiv. Nach dem Existenzsatz existiert das Minimum in (10).

Lokale Treue: Die Perturbationsmenge $\mathcal{P}_{\text{rot}}(\mathbf{x})$ enthält nach Satz 4.1 alle zyklischen Isomorphismen von $\mathbf{x}$. Die Gewichtsfunktion $\pi_{\mathbf{x}}^{\text{Graph}}$ maximiert das Gewicht für strukturell ähnliche Graphen. Damit misst L nach (3) tatsächlich die Abweichung in der lokalen Umgebung von $\mathbf{x}$ im Graphstruktur-Raum. ■ □

5.2 Komplexitätsanalyse von Graph-LIME

Satz 5.2 (Gesamtkomplexität von Graph-LIME). Graph-LIME hat eine Zeitkomplexität von $O(n^3 + n^2 \cdot p)$, wobei n die Graphgröße und p die Dimension des Merkmalsvektors ist.

Beweis. – *Perturbation:* n Rotationen à $O(n)$: $O(n^2)$.

– *Graph-Kernel:* Für jede Rotation eine LCS-Berechnung in $O(n^2)$: $n \cdot O(n^2) = O(n^3)$.

– *Lokales Training:* Gewichtete lineare Regression auf n Punkten mit p Features: $O(n \cdot p + p^2)$.

– *Gesamtlaufzeit:* $O(n^3 + n^2 \cdot p)$.

Dominanter Term: $O(n^3)$ für großes n bzw. $O(n^2 p)$ für großes p . ■ □

Abbildung 5 zeigt den Komplexitätsvergleich zwischen naiver Subgraph-Suche, dem Subgraph Algorithmus und Standard-LIME-Sampling sowie die Gesamtarchitektur von Graph-LIME.

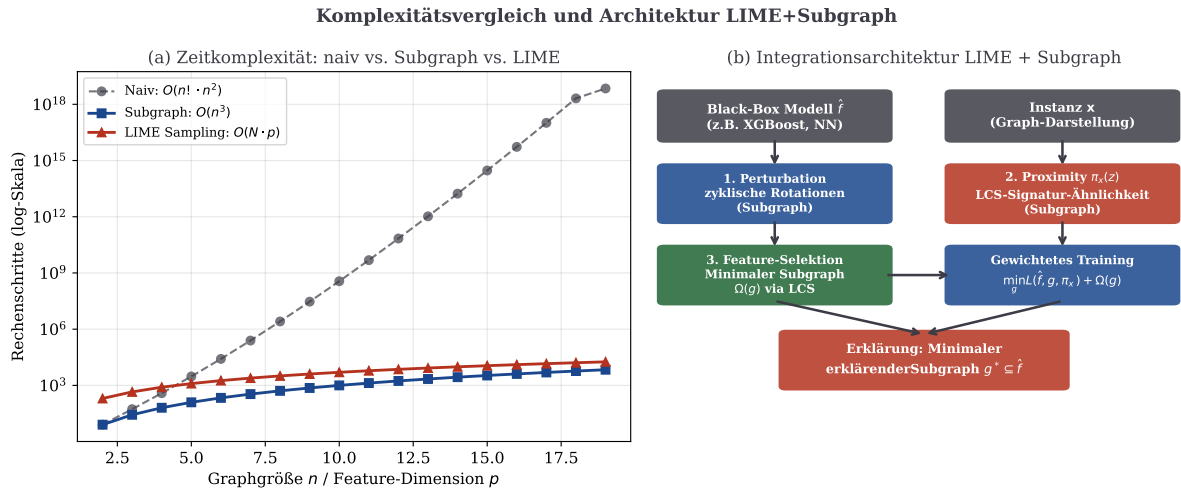


Abbildung 5: (a) Zeitkomplexität im Vergleich: Die naive Subgraph-Suche ($O(n! \cdot n^2)$, grau) ist für $n > 12$ praktisch nicht mehr handhabbar. Der Subgraph Algorithmus ($O(n^3)$, blau) wächst polynomial. Standard-LIME-Sampling ($O(N \cdot p)$, rot) wächst nur linear in n . (b) Integrationsarchitektur von Graph-LIME: Die drei Komponenten des Subgraph Algorithmus (Rotationsperturbation, LCS-Proximity, Subgraph-Inklusion) ersetzen die entsprechenden Standard-LIME-Komponenten für graphstrukturierte Daten.

6 Zusammenfassung

Die vorliegende Arbeit hat gezeigt, dass der Subgraph Algorithmus (Epp, 2026) LIME (Ribeiro et al., 2016) in drei fundamentalen Dimensionen erweitert:

Komponente	Standard-LIME	Graph-LIME
Perturbation	$\mathcal{N}(\mu, \sigma^2)$ Sampling	Zyklische Rotationen
Proximity-Maß	L2-Gaußkern	LCS-Signaturkern
Komplexität Ω	LASSO (ℓ_1 -Norm)	Subgraph-Inklusion
Erklärungsmodell	β -Koeffizienten	Minimaler Subgraph
Zeitkomplexität	$O(N \cdot p)$	$O(n^3 + n^2p)$
Datentypen	Beliebig	Graphstrukturiert

7 Ausblick

Die in dieser Arbeit vorgestellte Integration des Subgraph Algorithmus in LIME eröffnet eine Vielzahl von Forschungsrichtungen, die im Folgenden ausführlich diskutiert werden.

7.1 Erweiterung auf gerichtete und gewichtete Graphen

Die aktuelle Signaturformel (5) ist für ungewichtete Graphen definiert. Eine natürliche Erweiterung auf gewichtete Graphen der Form $G = (V, E, w)$ mit Gewichtsfunktion $w : E \rightarrow \mathbb{R}_{>0}$ könnte die Signatur wie folgt modifizieren:

$$\sigma_j^w = \sum_{i=0}^{n-1} w(v_i, v_j) \cdot r^i + j \cdot r^n,$$

wobei $r > \max_{e \in E} w(e)$ eine geeignete Basis ist, die Eindeutigkeit sicherstellt. Dies würde direkte Anwendungen in gewichteten sozialen Netzwerken, Transportnetzwerken und biochemischen Reaktionsgraphen ermöglichen.

Für gerichtete Graphen ist eine separate Behandlung von Eingangs- und Ausgangsgrad erforderlich:

$$\sigma_j^{\text{dir}} = \sigma_j^{\text{out}} + j^2 \cdot 2^{2n}, \quad \sigma_j^{\text{in}} = \sum_{i=0}^{n-1} A_{ji} \cdot 2^i + j \cdot 2^n.$$

Die LCS-Berechnung müsste dann auf beiden Signaturkomponenten gleichzeitig operieren.

7.2 Approximative LCS-Algorithmen für Echtzeit-XAI

Die LCS-Berechnung hat unter SETH eine untere Laufzeitschranke von $\Omega(n^{2-\epsilon})$ (Aboud et al., 2015). Für Echtzeit-Anwendungen wie interaktive Erklärungssysteme oder eingebettete Systeme (z. B. AUTOSAR-basierte Steuergeräte) ist $O(n^3)$ möglicherweise zu langsam.

Vielversprechende Ansätze umfassen:

- *Banded LCS* (Allison et al., 1986): Einschränkung der DP-Tabelle auf ein Band der Breite b , was $O(n \cdot b)$ ergibt. Dies ist sinnvoll, wenn Rotationen nahezu ausgerichtetes Graphen erwartet werden.

- *Locality-Sensitive Hashing (LSH) für LCS* (Indyk und Motwani, 1998): Approximative Ähnlichkeitssuche in subquadratischer Zeit.
- *Sketch-basierte Methoden*: Komprimierung der Signatursequenzen auf Vektoren fester Länge, auf denen dann euklidische Distanzen berechnet werden. Dieser Ansatz überbrückt die Lücke zwischen dem neuen Graph-Kernel und dem Standard-L2-Kernel.

7.3 Graph Neural Networks als Black-Box-Modell $\hat{f}$

Die wichtigste Anwendungsdomäne von Graph-LIME sind Erklärungen für *Graph Neural Networks* (GNNs) (Scarselli et al., 2009; Kipf und Welling, 2017). GNNs lernen Repräsentationen auf Graphen und werden zunehmend für Moleküldesign, Wissensgraphen und soziale Netzwerkanalyse eingesetzt.

- *GNNE explainer* (Ying et al., 2019) ist ein verwandtes Verfahren, das erklärende Subgraphen durch Maskierung lernt. Graph-LIME unterscheidet sich durch die kombinatorisch exakte Subgraph-Suche über Rotationen statt gradientenbasierter Maskierung.
- Eine direkte Integration mit *PyTorch Geometric* (Fey und Lenssen, 2019) und *Deep Graph Library* (Wang et al., 2019) wäre ein natürlicher nächster Schritt.
- Für molekulare GNNs (z. B. Vorhersage von Medikamentenwirksamkeit) könnte der minimale erklärende Subgraph direkt als erklärende Teilstruktur des Moleküls interpretiert werden.

7.4 Formalere Gütekriterien und Evaluationsmetriken

Die Evaluierung von XAI-Methoden ist ein offenes Forschungsproblem (Doshi-Velez und Kim, 2017). Für Graph-LIME müssen neue Metriken entwickelt werden:

- *Strukturelle Fidelität*: Wie gut approximiert der erklärende Subgraph G_g^* die Vorhersage des GNN in der lokalen Umgebung? Eine formale Definition könnte sein:

$$\text{Fidelity}(G_g^*, \hat{f}, \mathbf{x}) = 1 - \frac{|\hat{f}(\mathbf{x}) - g^*(G_g^*(\mathbf{x}))|}{|\hat{f}(\mathbf{x})|}.$$

- *Minimality*: Wie viele Kanten enthält der erklärende Subgraph im Vergleich zum vollständigen Modellgraphen? $\text{Minimality}(G_g^*) = 1 - |E_{G_g^*}|/|E_{G_f}|$.
- *Konsistenz*: Erhalten strukturell ähnliche Instanzen ähnliche erklärende Subgraphen? $\text{Consistency}(\mathbf{x}, \mathbf{x}') = \text{sim}_{\text{LCS}}(G_g^*(\mathbf{x}), G_g^*(\mathbf{x}'))$.

7.5 Parallelisierung der Rotationsberechnungen

Der dominante Term $O(n^3)$ der Graph-LIME-Komplexität entsteht durch n unabhängige LCS-Berechnungen. Nach Lemma 5.1 aus Epp (2026) sind diese im Allgemeinen unabhängig voneinander. Dies eröffnet direkte Parallelisierungsmöglichkeiten:

- *SIMD-Parallelisierung*: Auf modernen CPUs mit AVX-512-Instruktionen können mehrere LCS-DP-Tabellen gleichzeitig berechnet werden.
- *GPU-Parallelisierung*: Die n LCS-Berechnungen können auf n CUDA-Kerne verteilt werden, was die Laufzeit auf $O(n^2)$ amortisiert. Dies würde die Integration in AUTOSAR-Plattformen mit GPU-Co-Prozessoren (z. B. NVIDIA Drive) ermöglichen.
- *Verteilte Berechnung*: Für sehr große Graphen ($n > 10^4$) bietet sich eine Map-Reduce-Architektur an.

7.6 Integration in den AUTOSAR-Standard

Der Subgraph Algorithmus wurde ursprünglich für die Verifikation von Programmen durch Abstract Syntax Tree (AST)-Analyse entwickelt (Epp, 2026). Die natürliche Erweiterung auf AUTOSAR (AUTOSAR, 2023) als Softwarearchitekturstandard für automobiler Steuergeräte bietet folgende Anwendungen:

- *Erklärung von Sicherheitsentscheidungen*: Steuergeräte treffen sicherheitskritische Entscheidungen (Notbremsung, Lenkkorrektur). Graph-LIME könnte diese Entscheidungen durch minimale erklärende Subgraphen der Softwarekomponenten-Abhängigkeiten erklären.
- *Software-Äquivalenzprüfung*: Der Subgraph Algorithmus kann prüfen, ob zwei AUTOSAR-Softwarekomponenten strukturell äquivalent sind, was für Zertifizierungszwecke (ISO 26262) relevant ist.
- *Echtzeitfähigkeit*: Mit den in Abschnitt 7.2 genannten Approximationen könnte Graph-LIME auf AUTOSAR Adaptive Platform mit QNX-Echtzeitbetriebssystem implementiert werden.

7.7 Verbindung zu Shapley-Werten und SHAP

SHAP (SHapley Additive exPlanations) (Lundberg und Lee, 2017) ist eine weitere wichtige XAI-Methode, die auf Shapley-Werten aus der kooperativen Spieltheorie basiert. Für Graphen definiert man:

$$\phi_j = \sum_{S \subseteq V \setminus \{j\}} \frac{|S|! (|V| - |S| - 1)!}{|V|!} [v(S \cup \{j\}) - v(S)],$$

wobei $v(S)$ der Wert der Koalition S ist.

Eine interessante Forschungsfrage ist: Stimmt der minimale erklärende Subgraph G_g^* aus Graph-LIME mit dem Subgraphen der Knoten mit höchsten Shapley-Werten überein? Eine positive Antwort würde die theoretische Fundierung von Graph-LIME erheblich stärken.

7.8 Erweiterung auf dynamische Graphen

In vielen realen Anwendungen sind Graphen zeitlich veränderlich: Soziale Netzwerke, Verkehrsnetzwerke und biologische Signalwege ändern ihre Struktur über die Zeit. Eine

Erweiterung auf dynamische Graphen $G(t) = (V, E(t))$ könnte die Signaturformel (5) um eine Zeitkomponente erweitern:

$$\sigma_j(t) = \sum_{i=0}^{n-1} A_{ij}(t) \cdot 2^i + j \cdot 2^n + \lfloor t \rfloor \cdot 2^{2n}.$$

Graph-LIME würde dann nicht nur die räumliche, sondern auch die zeitliche Nachbarschaft einer Instanz berücksichtigen.

7.9 Anwendung auf bioinformatische Graphen

Proteininteraktionsnetzwerke, Metabolitenflüsse und Genregulationsnetzwerke sind natürliche Anwendungsfelder. Der minimale erklärende Subgraph für eine Krankheitsvorhersage könnte direkt als biologisch relevanter Pathway interpretiert werden:

- *Drogenresistenz*: Welcher minimale Teilpfad eines Metabolitnetzwerks erklärt die vorhergesagte Resistenz?
- *Krebsdiagnose*: Welche minimale Knotenmenge im Proteininteraktionsnetz unterscheidet Tumorzellen von Normalzellen?

Literaturverzeichnis

Literaturverzeichnis

- Abboud, A., Backurs, A., & Vassilevska Williams, V. (2015). Tight hardness results for LCS and other sequence similarity measures. *Proceedings of the 56th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 59–78.
- Allison, L., & Dillon, T. I. (1986). LCS on a simple SIMD machine. *Parallel Computing*, 3(4), 361–368.
- AUTOSAR Consortium. (2023). *AUTOSAR Adaptive Platform: Architecture and Specification Release 23-11*. AUTOSAR GbR, Munich.
- Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
- Epp, S. (2026). *Der Subgraph Algorithmus: Effiziente Graphvergleiche mittels Signatur-Arrays und zyklischer Rotationen*. Universität Bielefeld. <https://gitlab.com/epp-group/subgraph>.
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*.
- Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: Towards removing the curse of dimensionality. *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, 604–613.
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)*.

- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144.
- Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2009). The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1), 61–80.
- Wang, M., Zheng, D., Ye, Z., Gan, Q., Li, M., Song, X., . . . & Zhang, Z. (2019). Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*.
- Ying, Z., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). GNNExplainer: Generating explanations for graph neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.